

 Avaliação Final — Big Data para Negócios

Curso: Big Data para Negócios

Instituição: UFC - Universidade Federal do Ceará

Professor responsável: Luiz Alexandre Moreira Barros

Formato: Individual · Relatório + repositório de código + dashboard

Aluno(a): Gabriel Ramos e Thales Vieira

Data de entrega: _____

Etapa 4 - Modelo de Regressão

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    roc_auc_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)

# 1. LEITURA DA BASE

df = pd.read_csv(
    "/content/avaliacao_transactions.csv",
    parse_dates=["timestamp"]
)

print("\n===== ETAPA 4 - MODELO DE FRAUDE =====")
print(f"Total de registros: {len(df)}")
print(f"Total de fraudes: {df['is_fraud'].sum()}")
print(f"Taxa de fraude: {(df['is_fraud'].mean() * 100):.2f}%")

# 2. CRIAÇÃO DE VARIÁVEIS TEMPORAIS

df["hora"] = df["timestamp"].dt.hour
```

```
df["dia_semana"] = df["timestamp"].dt.dayofweek

# 3. DEFINIÇÃO DAS VARIÁVEIS

features = [
    "amount",
    "risk_score",
    "credit_score",
    "hora",
    "dia_semana",
    "transaction_type",
    "channel",
    "merchant_category",
    "segment"
]

target = "is_fraud"

X = df[features].copy()
y = df[target].astype(int)

# Variáveis numéricas
numeric_features = [
    "amount",
    "risk_score",
    "credit_score",
    "hora",
    "dia_semana"
]

# Variáveis categóricas
categorical_features = [
    "transaction_type",
    "channel",
    "merchant_category",
    "segment"
]

# 4. DIVISÃO ENTRE TREINO E TESTE

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=123,
    stratify=y
)

print("\n DIVISÃO DA BASE")
```

```
print("\n--- DIVISÃO DA BASE ---")
print(f"Registros de treino: {len(X_train)}")
print(f"Registros de teste: {len(X_test)}")
print(f"Fraudes no conjunto de teste: {y_test.sum()}")
```

5. PRÉ-PROCESSAMENTO

```
preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            StandardScaler(),
            numeric_features
        ),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                drop="first"
            ),
            categorical_features
        )
    ]
)
```

6. MODELO DE REGRESSÃO LOGÍSTICA

```
modelo = LogisticRegression(
    max_iter=2000,
    class_weight="balanced",
    random_state=123
)

pipeline = Pipeline(
    steps=[
        ("preprocessamento", preprocessor),
        ("modelo", modelo)
    ]
)
```

7. TREINAMENTO

```
pipeline.fit(X_train, y_train)

print("\nModelo treinado com sucesso.")
```

8. PROBABILIDADES E PREVISÕES

```
probabilidades = pipeline.predict_proba(X_test)[:. 1]
```

```
threshold = 0.50

previsoes = (
    probabilidades >= threshold
).astype(int)

# 9. MÉTRICAS

auc = roc_auc_score(
    y_test,
    probabilidades
)

matriz = confusion_matrix(
    y_test,
    previsoes
)

tn, fp, fn, tp = matriz.ravel()

precision = precision_score(
    y_test,
    previsoes,
    zero_division=0
)

recall = recall_score(
    y_test,
    previsoes,
    zero_division=0
)

f1 = f1_score(
    y_test,
    previsoes,
    zero_division=0
)

print("\n--- RESULTADOS DO MODELO ---")

print(f"AUC-ROC: {auc:.4f}")
print(f"Threshold utilizado: {threshold:.2f}")

print("\nMatriz de confusão:")
print(matriz)

print("\nLeitura da matriz de confusão:")
```

```
print(f"TN - legítimas classificadas corretamente: {tn}")
print(f"FP - legítimas sinalizadas como fraude: {fp}")
print(f"FN - fraudes não identificadas: {fn}")
print(f"TP - fraudes identificadas corretamente: {tp}")

print("\nMétricas da classe fraude:")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")

# 10. INTERPRETAÇÃO DOS COEFICIENTES

encoder = (
    pipeline
    .named_steps["preprocessamento"]
    .named_transformers_["cat"]
)

nomes_categoricos = encoder.get_feature_names_out(
    categorical_features
)

nomes_variaveis = (
    numeric_features
    + list(nomes_categoricos)
)

coeficientes = (
    pipeline
    .named_steps["modelo"]
    .coef_[0]
)

resultado_coeficientes = pd.DataFrame({
    "variavel": nomes_variaveis,
    "coeficiente": coeficientes
})

# Odds Ratio
resultado_coeficientes["odds_ratio"] = np.exp(
    resultado_coeficientes["coeficiente"]
)

# Valor absoluto para ordenar importância
resultado_coeficientes["abs_coef"] = (
    resultado_coeficientes["coeficiente"]
    .abs()
)
```

```
resultado_coeficientes = (
    resultado_coeficientes
    .sort_values(
        "abs_coef",
        ascending=False
    )
)

print("\n--- COEFICIENTES DE MAIOR MAGNITUDE ---")

print(
    resultado_coeficientes[
        [
            "variavel",
            "coeficiente",
            "odds_ratio"
        ]
    ]
    .head(15)
    .round(4)
    .to_string(index=False)
)

# 11. MAIORES COEFICIENTES POSITIVOS

positivos = (
    resultado_coeficientes[
        resultado_coeficientes["coeficiente"] > 0
    ]
    .sort_values(
        "coeficiente",
        ascending=False
    )
)

print("\n--- VARIÁVEIS QUE MAIS AUMENTAM A CHANCE ESTIMADA DE FRAUDE")

print(
    positivos[
        [
            "variavel",
            "coeficiente",
            "odds_ratio"
        ]
    ]
    .head(10)
    .round(4)
    .to_string(index=False)
)
```

```

        .to_string(index=False,
    )

# 12. MAIORES COEFICIENTES NEGATIVOS

negativos = (
    resultado_coeficientes[
        resultado_coeficientes["coeficiente"] < 0
    ]
    .sort_values(
        "coeficiente",
        ascending=True
    )
)

print("\n--- VARIÁVEIS QUE MAIS REDUZEM A CHANCE ESTIMADA DE FRAUDE")

print(
    negativos[
        [
            "variavel",
            "coeficiente",
            "odds_ratio"
        ]
    ]
    .head(10)
    .round(4)
    .to_string(index=False)
)

print("\n===== FIM DA ETAPA 4 =====")

```

Total de registros: 30000

Total de fraudes: 751

Taxa de fraude: 2.50%

--- DIVISÃO DA BASE ---

Registros de treino: 24000

Registros de teste: 6000

Fraudes no conjunto de teste: 150

Modelo treinado com sucesso.

--- RESULTADOS DO MODELO ---

AUC-ROC: 0.7673

Threshold utilizado: 0.50

Matriz de confusão:

```
[[4267 1583]
 [ 49  101]]
```

Leitura da matriz de confusão:

TN – legítimas classificadas corretamente: 4267

FP – legítimas sinalizadas como fraude: 1583

FN – fraudes não identificadas: 49

TP – fraudes identificadas corretamente: 101

Métricas da classe fraude:

Precision: 0.0600

Recall: 0.6733

F1-score: 0.1101

--- COEFICIENTES DE MAIOR MAGNITUDE ---

	variavel	coeficiente	odds_ratio
	segment_Premium	-2.5350	0.0793
	segment_Standard	-1.3742	0.2530
	channel_pos	-0.9997	0.3680
	merchant_category_viagem	0.9615	2.6156
	channel_web	-0.8819	0.4140
	channel_atm	-0.6624	0.5156
	transaction_type_transferencia	0.3280	1.3881
	merchant_category_saude	0.2730	1.3138
	transaction_type_saque	0.2470	1.2802
	hora	-0.1496	0.8611
	merchant_category_eletronico	-0.1455	0.8646
	risk_score	0.0881	1.0920
	dia_semana	0.0763	1.0793
	merchant_category_servicos	-0.0656	0.9365
	credit_score	0.0624	1.0644

--- VARIÁVEIS QUE MAIS AUMENTAM A CHANCE ESTIMADA DE FRAUDE ---

	variavel	coeficiente	odds_ratio
	merchant_category_viagem	0.9615	2.6156
	transaction_type_transferencia	0.3280	1.3881
	merchant_category_saude	0.2730	1.3138
	transaction_type_saque	0.2470	1.2802
	risk_score	0.0881	1.0920
	dia_semana	0.0763	1.0793
	credit_score	0.0624	1.0644
	merchant_category_varejo	0.0106	1.0106
	transaction_type_pagamento	0.0001	1.0001

